

Rapport de Projet — **MEDISCAN AI**

Plateforme de recherche multimodale d'images médicales par le contenu (**CBIR**)

L3X1 — Université Paris Cité



*Le présent document est le **rapport de projet** de **MEDISCAN AI**, prototype académique non clinique de recherche d'images médicales par le contenu. Il couvre l'ensemble des travaux de l'équipe L3X1 : besoins, conception, implémentation, tests et déploiement.*

Informations document

Version : v1.0
Référence : Rapport de Projet MEDISCAN AI v1.0
Confidentialité : Interne projet (usage universitaire)
Encadrant : Camille Kurtz
Équipe : L3X1 (Taskin Ozan, Rayane Taouache, Ales Ferhani, Maxime Huang)

Équipe projet

L'équipe **L3X1** est composée de quatre étudiants de l'Université Paris Cité. Chaque membre a contribué à plusieurs pôles en fonction de ses compétences.



Taskin Ozan

AI / Frontend



Ferhani Ales

AI / Test



Taouache Rayane

Backend / Test



Huang Maxime

Frontend / Backend

IA / Frontend

Taskin Ozan

Architecture du système, pipeline CBIR, fine-tuning BioMedCLIP, interface React.

IA / Test

Ferhani Ales

Scripts d'évaluation, stratégie de tests, traitement embeddings.

Backend / Test

Taouache Rayane

API FastAPI, services backend, déploiement VPS, intégration MongoDB, tests d'intégration.

Frontend / Backend

Huang Maxime

Interface React, configuration Nginx, tests frontend.

Le projet a été développé **collectivement** : chaque membre a contribué à l'ensemble des composants du système. Naturellement, au fil du projet, des expertises se sont constituées au sein de l'équipe, reflétées dans la répartition des rôles ci-dessus.

Table des matières

Équipe projet	2
Guide de lecture	6
Résumé exécutif	7
1 Contexte et enjeux	8
1.1 Contexte métier et motivation	8
1.2 Problématique	8
1.3 Notre approche	8
2 Benchmark et positionnement	9
2.1 Recherche d'images par le contenu (CBIR)	9
2.2 Solutions existantes et analyse comparative	9
2.3 Un terrain d'opportunité pour la recherche	9
3 Gestion de projet	10
3.1 Méthodologie	10
3.2 Planning et jalons	10
4 Produit	11
4.1 Réponse à la problématique	11
4.2 Cas d'utilisation	11
4.3 Interface et ergonomie	12
4.4 Modes de recherche	13
4.5 Résultats et exploration	14
5 Technologies clés	16
5.1 Architecture des pipelines d'embedding	16
5.2 Technologies de développement	17
5.3 FAISS et optimisation de la recherche	17
5.4 Stockage cloud et infrastructure d'entraînement	17
6 Déploiement et exploitation	18
6.1 Architecture et pipeline de déploiement	18
6.2 Production, accessibilité et évolutivité	18
7 Évaluation et résultats	19
7.1 Résultats du moteur visuel (DINOv2)	19
7.2 Résultats du moteur sémantique (BioMedCLIP)	19

7.3	Tests unitaires	20
7.4	Performance technique	20
7.5	Limites observées et synthèse	20
8	Difficultés rencontrées	21
8.1	Fine-tuning et choix des modèles	21
8.2	Manipulation d'un dataset volumineux	21
8.3	Absence de benchmark de référence en CBIR médical	21
9	Bilan et retour d'expérience	22
10	Perspectives	23
	Références bibliographiques	24
A	Guide d'installation et de déploiement	25
B	Documentation API REST	25
C	Captures d'écran de l'application	26
D	Extraits de code complets	26
E	Résultats de tests détaillés	26
F	Glossaire	26

Table des figures

1	Planning du projet MEDISCAN AI	10
2	Page d'accueil de MEDISCAN AI	12
3	Sélecteur de thème et de langue	12
4	Hub de sélection du mode de recherche	12
5	Mode Visual Analysis	13
6	Mode Interpretive Analysis	13
7	Mode Text Query	13
8	Grille de résultats avec filtres et scores de similarité	14
9	Relance de recherche depuis une sélection d'images	14
10	Comparaison entre l'image requête et les résultats retournés	15
11	Synthèse exploratoire générée par IA	15
12	Architecture des deux pipelines d'embedding — BioMedCLIP (sémantique) et DINOv2 (visuel)	16
13	DINOv2 : Precision@k	19
14	DINOv2 : Hit@k	19
15	BioMedCLIP : Precision@k	19
16	BioMedCLIP : Hit@k	19

Liste des tableaux

Guide de lecture

Le présent **rapport de projet** documente l'intégralité du cycle de réalisation de **ME-DISCAN AI**. Il adopte une structure professionnelle couvrant la gestion de projet, la conception, la réalisation et l'exploitation du système.

Manager ou décideur - Vision stratégique

Pour comprendre le projet et sa valeur ajoutée sans entrer dans les détails techniques :

- **Résumé exécutif** - synthèse du projet en une page
- **Chapitre 1** - contexte, enjeux métier et objectifs
- **Chapitre 9** - bilan, retour sur investissement et perspectives

Encadrant technique - Architecture et réalisation

Pour évaluer les choix techniques et la qualité de l'implémentation :

- **Chapitre 2** - benchmark et justification des technologies retenues
- **Chapitre 5** - architecture globale, pipeline CBIR, modèle de données
- **Chapitre 6** - réalisation et extraits de code significatifs
- **Chapitre 8** - déploiement, configuration et maintenance

Encadrant pédagogique - Rigueur et méthode

Pour évaluer la démarche projet et la conformité aux exigences :

- **Chapitre 3** - gestion de projet, méthodologie et planning
- **Chapitre 4** - cahier des charges, besoins fonctionnels et non fonctionnels
- **Chapitre 7** - recette, stratégie de tests et gestion des risques

Lecteur externe - Fonctionnalités et positionnement

Pour comprendre ce que fait le système et comment il se positionne :

- **Chapitre 1** - contexte et enjeux
- **Chapitre 2** - benchmark et positionnement concurrentiel
- **Annexe C** - captures d'écran de l'application

Le **glossaire** (Annexe F) définit les termes spécialisés du rapport. Les **références** précisent les sources utilisées. L'**index** (en fin de document) localise les mots-clés principaux.

Résumé exécutif

MEDISCAN AI est une plateforme de recherche d'images médicales par le contenu, développée par l'équipe L3X1 à l'Université Paris Cité. Elle est **aujourd'hui en production** sur <https://mediscan-ai.site>.

Contexte

Les outils actuels, notamment les systèmes **PACS**, reposent sur des mots-clés et ne permettent pas de retrouver un cas similaire à partir d'une image de référence. Les Vision Transformers récents ouvrent la voie à une recherche par similarité visuelle et sémantique.

Solution

- Moteur de recherche combinant deux approches complémentaires : similarité visuelle et analyse du contenu médical.
- Trois modes d'usage : **par image** (analyse visuelle), **par image** (sens clinique) et **par description textuelle**.
- Une synthèse automatique résume les résultats et met en évidence les tendances communes.

Architecture

- **Confidentialité** : aucune donnée ne quitte l'infrastructure de l'équipe.
- **Accessibilité** : interface web sans installation, disponible 24h/24.
- **Couverture** : index de **80 000 images médicales** référencées.

Résultats

Le système retrouve des cas pertinents et exploitables dans plus de **88 %** des situations, validé sur un jeu de **9 000 requêtes** couvrant l'ensemble des modes de recherche.

Cadre d'usage

MEDISCAN AI est un prototype académique non clinique : il ne pose aucun diagnostic et ne remplace pas un professionnel de santé.

Perspectives

- Intégration native du format **DICOM** et fine-tuning sur données internes.
- Mise en place d'un pipeline **CI/CD** pour automatiser les déploiements.

1 Contexte et enjeux

Cette section présente le cadre dans lequel s'inscrit **MEDISCAN AI**, la problématique qu'il adresse et l'approche retenue par l'équipe.

1.1 Contexte métier et motivation

La recherche d'images médicales par le contenu visuel est l'un des apports concrets des nouvelles technologies appliquées à la santé. Elle ouvre des perspectives utiles à différents profils d'utilisateurs.

- Un médecin peut retrouver rapidement des cas antérieurs similaires pour appuyer son analyse.
- Un étudiant peut explorer un corpus d'images sous différents angles pour enrichir sa formation.
- Un chercheur peut naviguer efficacement dans de grands ensembles d'images médicales.

1.2 Problématique

Comment permettre aux parties prenantes de la santé d'interroger un corpus d'images médicales à la fois par le contenu visuel et par une description textuelle, tout en garantissant la pertinence des résultats ?

1.3 Notre approche

Plutôt que de s'appuyer uniquement sur des mots-clés, nous avons fait le choix d'une recherche multimodale : l'utilisateur peut interroger le système par l'image ou par une description textuelle, selon son besoin.

- Deux moteurs complémentaires couvrent la similarité visuelle et le sens clinique.
- L'interface permet de comparer, filtrer et affiner les résultats sans relancer de recherche.
- Une synthèse automatique aide à interpréter les résultats retournés.

2 Benchmark et positionnement

Cette section situe **MEDISCAN AI** face aux solutions existantes en imagerie médicale.

2.1 Recherche d'images par le contenu (CBIR)

Le **CBIR** (*Content-Based Image Retrieval*) désigne la recherche d'images par leur contenu plutôt que par mots-clés. Les approches modernes fondées sur l'apprentissage profond permettent de capturer à la fois la similarité visuelle et le sens clinique d'une image.

2.2 Solutions existantes et analyse comparative

Comparaison de **MEDISCAN AI** avec les systèmes de **CBIR médical** et les principales plateformes d'imagerie médicale connues.

Système	Recherche visuelle	Recherche sémantique	Recherche textuelle	Comparaison	Synthèse IA	Interface produit	Confidentialité
MEDISCAN AI	✓	✓	✓	✓	✓	✓	✓
Yottixel	✓	×	×	×	×	✓	✓
OpenI (NLM)	×	×	✓	×	×	✓	×
MedGemma	×	×	✓	×	✓	✓	×
MONAI (NVIDIA)	×	×	×	×	×	✓	✓
Aidoc	×	×	×	×	✓	✓	×

2.3 Un terrain d'opportunité pour la recherche

Les systèmes **PACS** déployés dans les hôpitaux ne disposent pas de recherche par le contenu visuel : les requêtes restent limitées aux métadonnées textuelles. L'intégration d'un moteur multimodal comme **MEDISCAN AI** constitue ainsi un terrain d'expérimentation prometteur pour la recherche en imagerie médicale.

3 Gestion de projet

Cette section présente la méthodologie de travail adoptée par l'équipe L3X1 et le planning du projet.

3.1 Méthodologie

L'équipe a adopté un cycle **incrémental** inspiré des méthodes agiles.

- **Incréments** : quatre livrables successifs : cœur CBIR, API backend, interface frontend, fine-tuning et évaluation.
- **Versionnement** : branches de fonctionnalité sur **SVN**, vérification avant chaque fusion sur la branche principale.
- **Coordination** : points de synchronisation bi-quotidiens sur **Discord**.

3.2 Planning et jalons

Le projet s'est déroulé sur un semestre, structuré en quatre phases successives.

- **Phase 1** : analyse des besoins, choix techniques et mise en place de l'environnement de développement.
- **Phase 2** : implémentation du cœur CBIR : embedders, index FAISS, pipeline de recherche et API backend.
- **Phase 3** : développement de l'interface React, intégration frontend-backend et fine-tuning de BioMedCLIP sur ROCov2.
- **Phase 4** : évaluation, tests, déploiement VPS et rédaction du rapport.

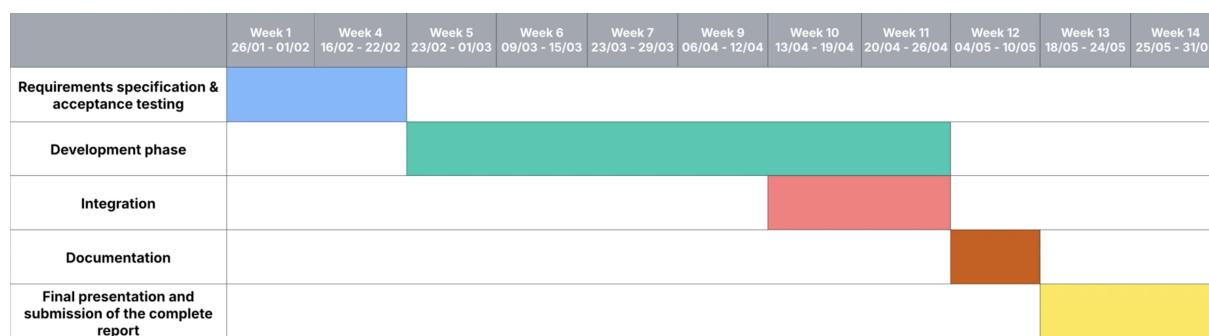


FIGURE 1 – Planning du projet MEDISCAN AI

4 Produit

Cette section présente **MEDISCAN AI** en tant que produit déployé : sa réponse concrète à la problématique posée et les cas d'usage rendus possibles par la version aujourd'hui en ligne.

4.1 Réponse à la problématique

Comment permettre aux parties prenantes de la santé d'interroger un corpus d'images médicales à la fois par le contenu visuel et par une description textuelle, tout en garantissant la pertinence des résultats ?

MEDISCAN AI répond à cette problématique sur trois axes complémentaires.

- **Rapidité** : optimisations sur l'index vectoriel et le traitement des données pour des réponses en quelques centaines de millisecondes.
- **Pertinence** : un corpus de référence de **80 000 images** issues du dataset public **ROCOv2**, couvrant un large spectre de modalités et d'organes.
- **Interopérabilité / Confidentialité** : architecture pluggable, capable d'indexer n'importe quel corpus interne, déployable localement ou exposée par API, sans transfert vers un service externe.

4.2 Cas d'utilisation

La plateforme est aujourd'hui en ligne sur <https://mediscan-ai.site>. Elle permet, depuis n'importe quel navigateur et sans installation, plusieurs usages concrets autour de la recherche d'images médicales.

- **Recherche par image** : retrouver les cas visuellement ou cliniquement proches d'une image requête.
- **Recherche textuelle** : interroger le corpus à partir d'une description clinique.
- **Comparaison d'images** : confronter côte à côte l'image requête et les résultats retournés.
- **Synthèse exploratoire** : obtenir une interprétation automatique des résultats à des fins d'analyse.
- **Export des résultats** : récupérer les résultats au format JSON, CSV ou PDF.

4.3 Interface et ergonomie

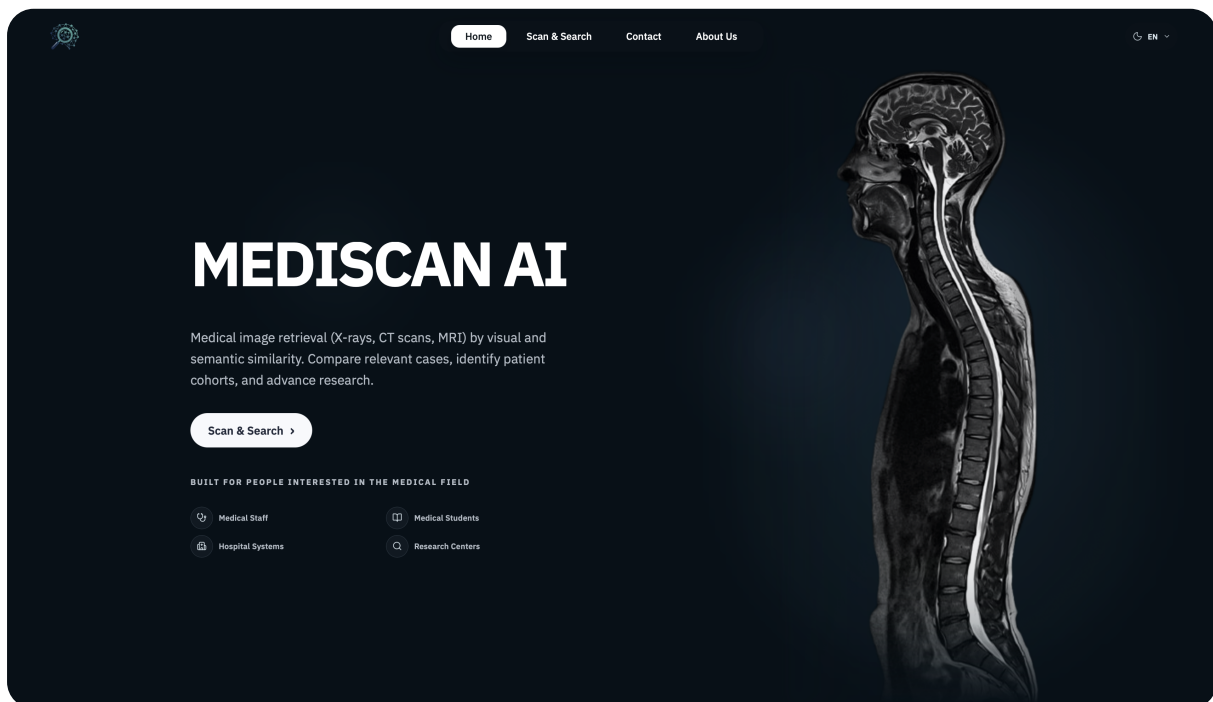


FIGURE 2 – Page d'accueil de MEDISCAN AI

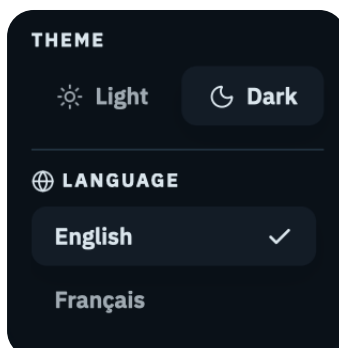


FIGURE 3 – Sélecteur de thème et de langue

L'interface propose un **thème clair et sombre** adaptable à l'environnement de travail, ainsi qu'une bascule de langue **FR / EN** accessible depuis la barre de navigation.

Le **hub central** est le point d'entrée vers les deux pages dédiées aux modes de recherche : Visual Search pour la recherche par image, et Text Guided Search pour la recherche textuelle.

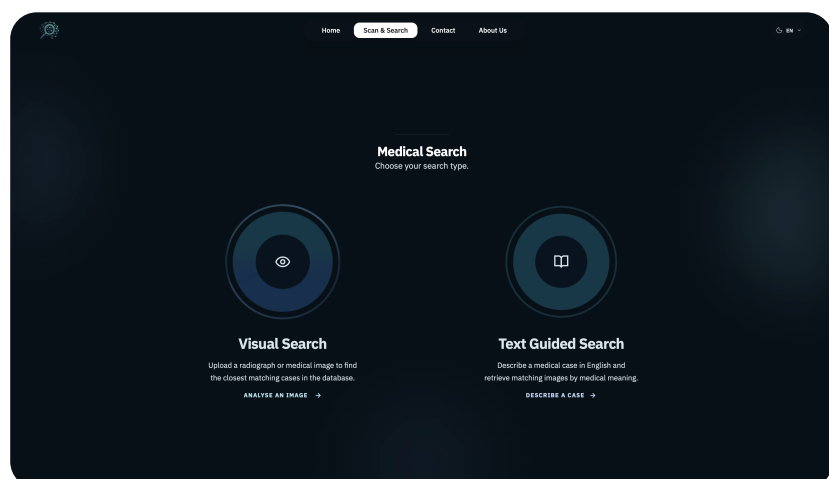
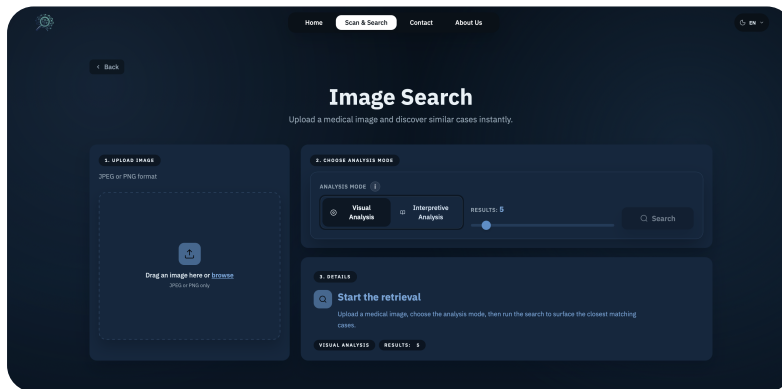


FIGURE 4 – Hub de sélection du mode de recherche

4.4 Modes de recherche

L'application propose trois modes complémentaires accessibles depuis le hub central.



L'utilisateur dépose une image médicale. Le système retourne les images visuellement proches via **DINOv2** (768D). Adapté à la recherche par morphologie et structure anatomique.

FIGURE 5 – Mode Visual Analysis

Recherche par proximité médicale via **BioMedCLIP** fine-tuné (512D). Privilégie le sens clinique sur la ressemblance visuelle.

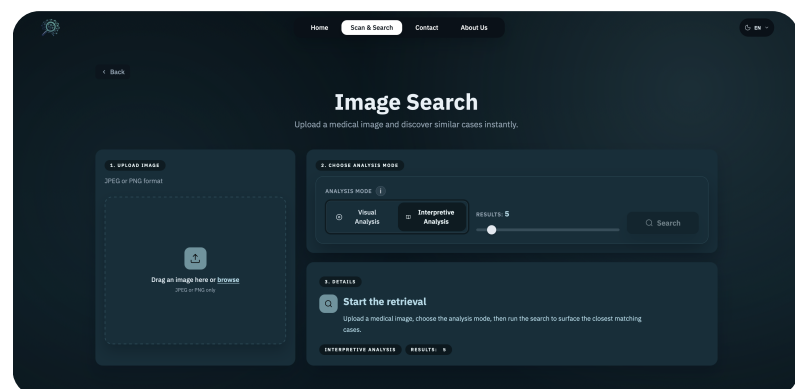
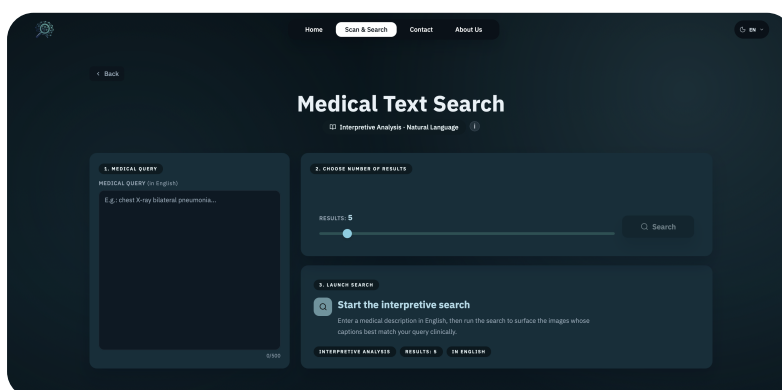


FIGURE 6 – Mode Interpretive Analysis



L'utilisateur saisit une description clinique. Le système retourne les images alignées via **BioMedCLIP** (500 caractères max).

FIGURE 7 – Mode Text Query

4.5 Résultats et exploration

Une fois la recherche lancée, l'interface propose des outils d'exploration et de relance pour affiner les résultats.

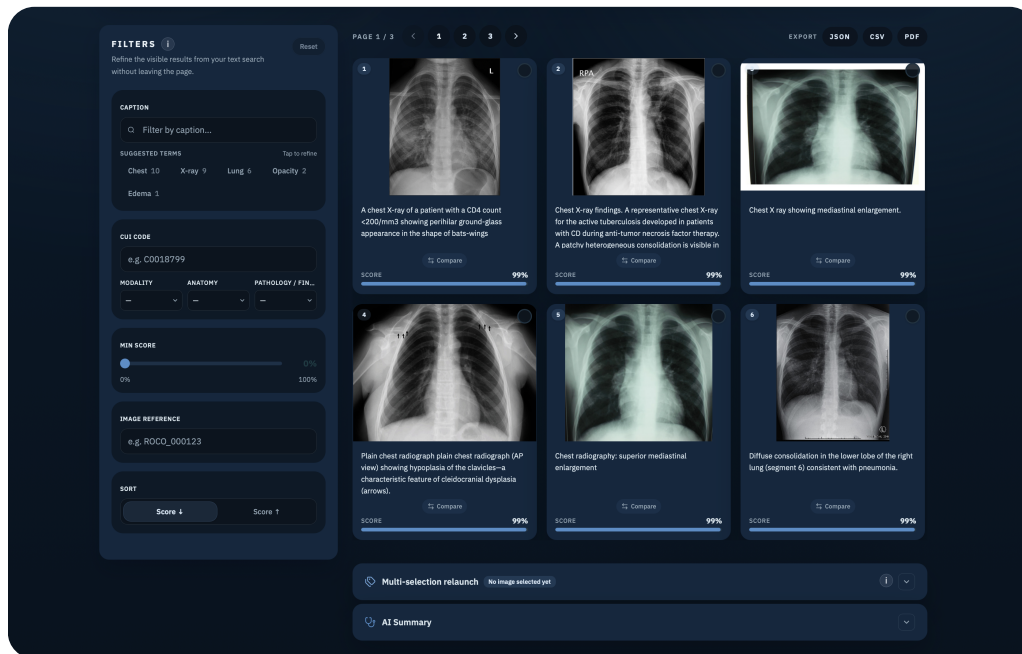


FIGURE 8 – Grille de résultats avec filtres et scores de similarité

Les filtres affinent les résultats sans relancer la recherche : **score**, **caption**, **CUI** et **tri**.

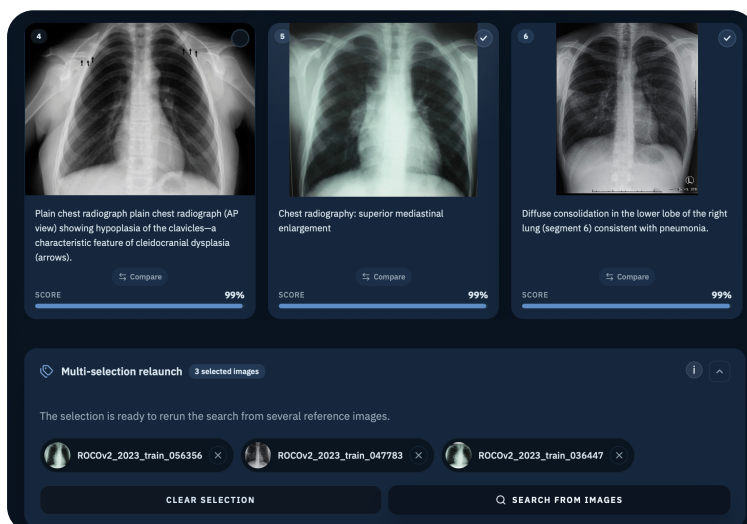


FIGURE 9 – Relance de recherche depuis une sélection d'images

Une **sélection d'images** devient une **nouvelle requête** via **centroïde d'embeddings**.

- [1] Sélectionner une ou plusieurs images.
- [2] Relancer depuis la sélection.

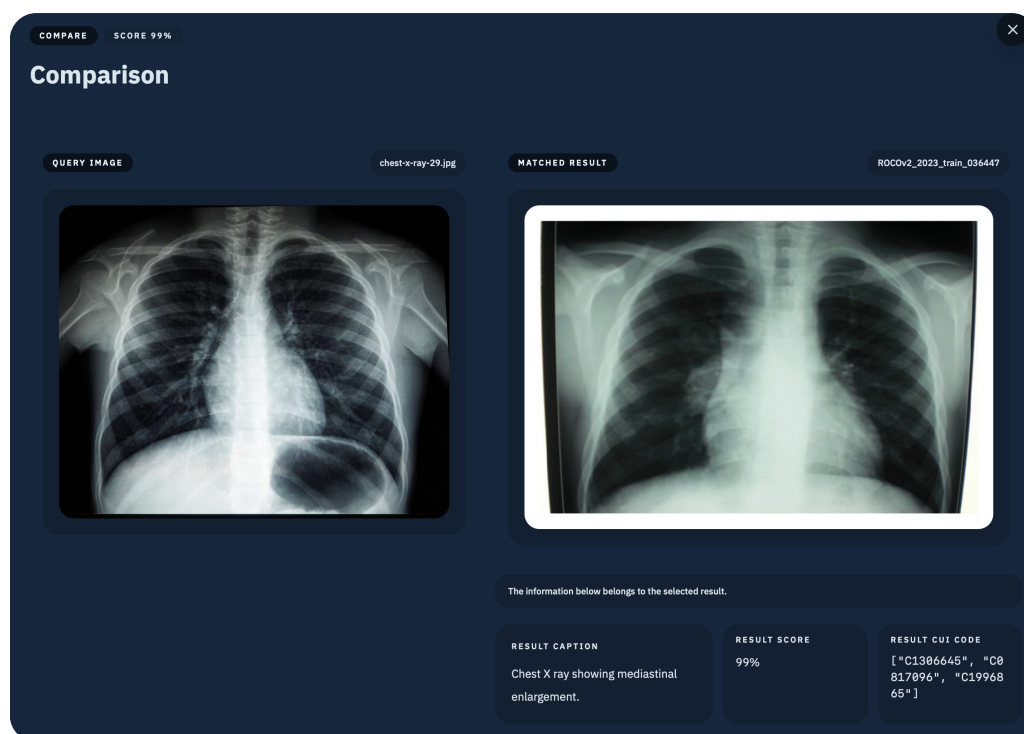


FIGURE 10 – Comparaison entre l'image requête et les résultats retournés

L'interface affiche **côte à côte** l'**image requête** et les **résultats retournés**, permettant une **comparaison visuelle directe**.

Le **module de synthèse** analyse les résultats et génère une **interprétation exploratoire** assistée par IA.

- [1] Synthèse générée via **Groq** à partir des captions des résultats.
- [2] Identifie les **patterns récurrents** sans poser de diagnostic.

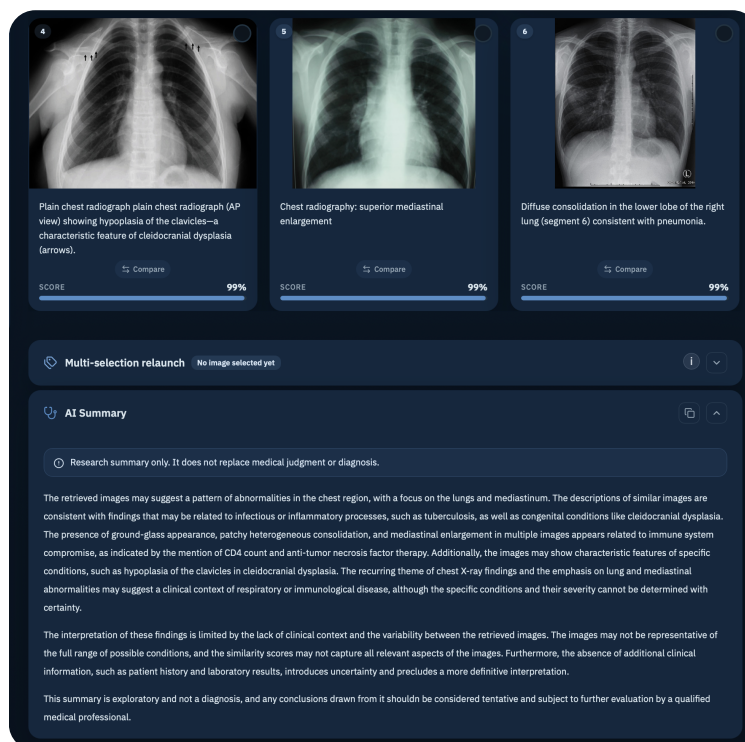


FIGURE 11 – Synthèse exploratoire générée par IA

5 Technologies clés

MEDISCAN AI s'appuie sur deux modèles d'IA aux architectures complémentaires, combinés à un moteur de recherche vectorielle haute performance.

5.1 Architecture des pipelines d'embedding

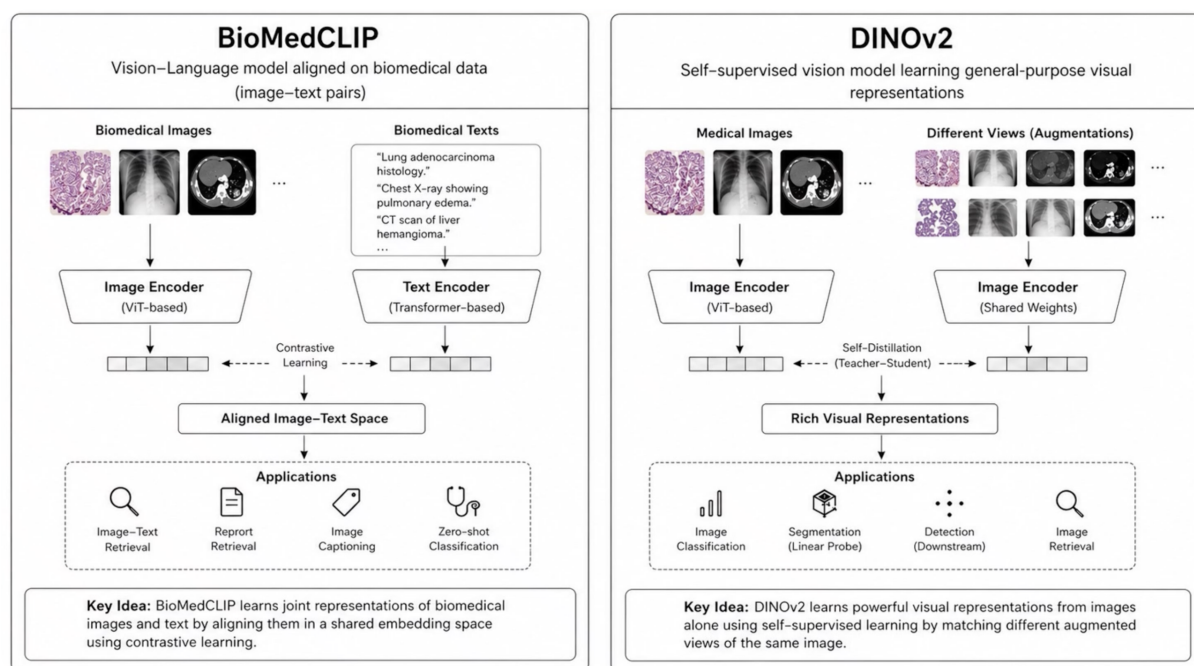


FIGURE 12 – Architecture des deux pipelines d'embedding — BioMedCLIP (sémantique) et DINOv2 (visuel)

BioMedCLIP (512D)

Architecture duale : un encodeur image (ViT) et un encodeur texte (Transformer) entraînés conjointement sur 15 millions de paires image-texte biomédicales.

- Aligne images et textes dans un espace vectoriel commun.
- Utilisé pour les modes **Interpretive Analysis** et **Text Guided Search**.

DINOv2 (768D)

Vision Transformer (ViT-L/14) pré-entraîné de manière auto-supervisée sur 142 millions d'images sans annotation.

- Capture structures visuelles, textures et morphologie.
- Utilisé pour le mode **Visual Search**.

Fine-tuning de BioMedCLIP. Le modèle a été ré-entraîné sur le dataset **ROCOv2** sur plusieurs epochs afin d'adapter ses représentations sur **ROCOv2**.

5.2 Technologies de développement

Stack moderne organisée en trois blocs : frontend, backend et couche IA.



5.3 FAISS et optimisation de la recherche

La recherche vectorielle repose sur **FAISS** (*Facebook AI Similarity Search*).

- Index **FlatIP** sur **80 000 vecteurs** : recherche exacte en quelques ms.
- Chargement *lazy* des modèles et de l'index pour limiter la mémoire.
- Architecture *stateless* : passage à l'échelle vers plusieurs millions d'images.

5.4 Stockage cloud et infrastructure d'entraînement

Deux ressources externes complémentaires : **Hugging Face** pour les données, **Kaggle** pour le calcul.



Hugging Face : Stockage cloud

Plateforme essentielle qui héberge l'ensemble des **80 000 images** du dataset **ROCOv2**. Elle permet un accès à la demande, sans dupliquer le corpus en local, tout en garantissant l'unicité de la version utilisée par l'équipe et un déploiement allégé.



Kaggle : GPU d'entraînement

Ressource clé pour le **fine-tuning de BioMedCLIP**. Les GPU mis à disposition gratuitement par Kaggle ont permis d'entraîner les modèles sans investir dans une infrastructure matérielle dédiée, ouvrant la voie à des itérations rapides.

6 Déploiement et exploitation

Cette section présente l'infrastructure mise en place pour rendre **MEDISCAN AI** accessible en production, ainsi que les choix garantissant sa stabilité et son évolutivité.

6.1 Architecture et pipeline de déploiement

L'application est déployée sur un **VPS Hostinger**, accessible via `mediscan-ai.site`.

- **Nginx** - reverse proxy, routage des requêtes frontend et backend.
- **Gunicorn + Uvicorn** - serveur ASGI pour l'API FastAPI.
- **systemd** - gestion des services et redémarrage automatique.

Le déploiement suit un pipeline en quatre étapes.

- [1] Configuration du serveur Linux : mises à jour, firewall, ports 80/443.
- [2] Installation des dépendances backend et build du frontend (Vite).
- [3] Configuration Nginx avec virtual host et certificat SSL.
- [4] Activation des services systemd pour le backend et le frontend.

6.2 Production, accessibilité et évolutivité

MEDISCAN AI est déployé en production et utilisable depuis n'importe quel navigateur, sans installation côté utilisateur.

- **Accessibilité** - le service est disponible 24h/24 via `mediscan-ai.site`, avec connexion chiffrée HTTPS.
- **Confidentialité** - aucune image n'est transmise à un service externe ; tout le traitement s'effectue sur le serveur de l'équipe.
- **Stabilité** - le redémarrage automatique (systemd) garantit la continuité du service en cas d'arrêt inattendu.

L'architecture choisie permet une montée en charge progressive.

- L'index FAISS est conçu pour passer de 80 000 à plusieurs millions d'images sans modifier l'interface ni l'API.
- Le backend FastAPI est sans état (*stateless*) : plusieurs instances peuvent être déployées en parallèle derrière Nginx pour absorber la charge.
- L'ajout d'un nouveau mode de recherche ou d'un nouveau modèle d'embedding ne nécessite pas de refonte de l'architecture.

7 Évaluation et résultats

L'équipe a pu s'appuyer sur des **annotations manuelles** produites par des **doctorants en imagerie médicale**, fournissant une vérité terrain experte (**modalité, organe, région anatomique**). Plus de **9 000 requêtes** ont ainsi été évaluées sur les deux moteurs.

7.1 Résultats du moteur visuel (DINOv2)

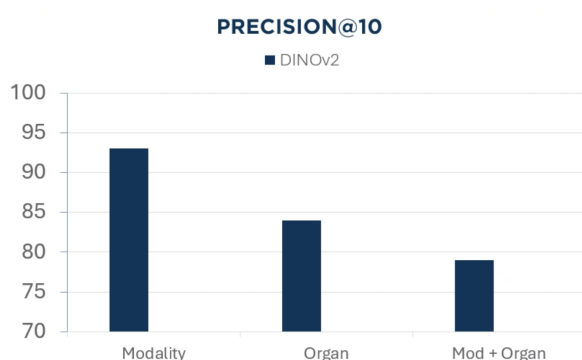


FIGURE 13 – DINOv2 : Precision@k

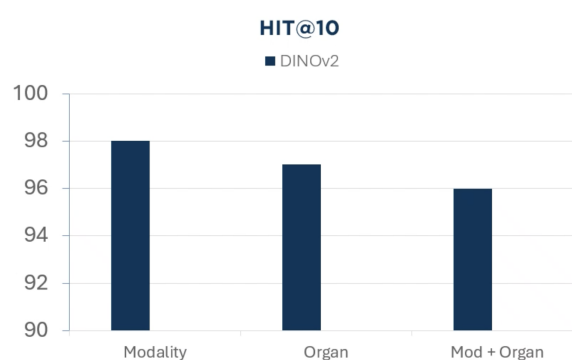


FIGURE 14 – DINOv2 : Hit@k

Interprétation. DINOv2 obtient une **Precision** élevée sur la modalité et l'anatomie, confirmant qu'il capture efficacement les structures visuelles. Le **Hit@k** valide son usage pour la recherche par similarité visuelle.

7.2 Résultats du moteur sémantique (BioMedCLIP)

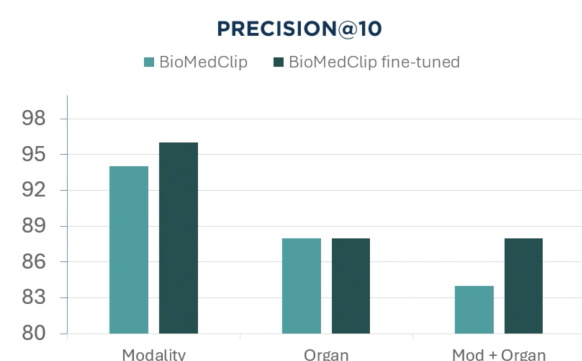


FIGURE 15 – BioMedCLIP : Precision@k

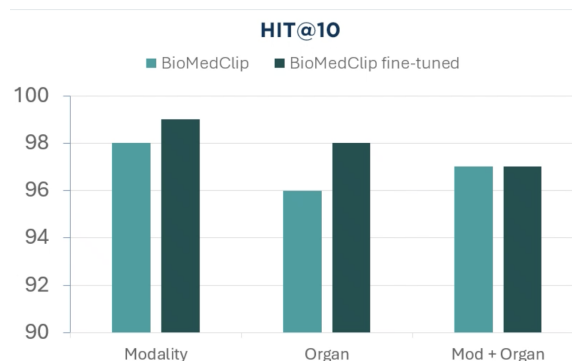


FIGURE 16 – BioMedCLIP : Hit@k

Interprétation. BioMedCLIP capte mieux le **sens clinique** grâce à son alignement image-texte fine-tuné sur ROCov2. Ses scores combinant modalité et organe sont supérieurs à DINOv2, validant son usage pour la recherche sémantique.

7.3 Tests unitaires

Une stratégie de tests **unitaires** et **d'intégration** a été mise en place pour garantir la fiabilité du backend et la stabilité du pipeline de recherche.

- **Backend** : tests `pytest` couvrant les neuf endpoints REST, validation des entrées, formats de réponse, gestion des erreurs.
- **Pipeline CBIR** : tests sur les embedders, le chargement des index **FAISS** et la cohérence des résultats de recherche.
- **Intégration** : scénarios bout-en-bout simulant une requête utilisateur depuis l'upload d'image jusqu'au retour des résultats.

7.4 Performance technique

Au-delà de la pertinence des résultats, le système a été évalué sur ses **performances en production**.

- **Temps de réponse** : recherche vectorielle exécutée en **quelques centaines de millisecondes** sur l'index de 80 000 vecteurs.
- **Empreinte mémoire** : chargement *lazy* des modèles et de l'index pour limiter la consommation au démarrage.
- **Scalabilité** : architecture *stateless* permettant de déployer plusieurs instances backend derrière Nginx pour absorber la charge.

7.5 Limites observées et synthèse

- **Modalités rares** : les modalités sous-représentées dans ROCov2 obtiennent des scores plus faibles que les modalités majoritaires.
- **Captions ambiguës** : la recherche textuelle est sensible à la qualité et à la précision de la description fournie.
- **Dépendance au dataset** : les performances reflètent le périmètre de ROCov2 et ne préjugent pas du comportement sur d'autres corpus médicaux.

Synthèse. Malgré ces limites, les résultats valident la pertinence de **MEDISCAN AI** : les deux moteurs sont **complémentaires** (DINOv2 pour la proximité visuelle, BioMedCLIP pour le sens clinique), les tests garantissent la fiabilité du système et les performances permettent un usage fluide en production.

8 Difficultés rencontrées

Cette section revient sur les principales difficultés rencontrées au cours du projet.

8.1 Fine-tuning et choix des modèles

Plusieurs candidats existaient pour la partie sémantique (BioMedCLIP, PubMedCLIP, MedCLIP).

- **Choix du modèle** : sélection de **BioMedCLIP** pour son alignement image-texte biomédical.
- **Fine-tuning** : adaptation à **ROCOv2** avec plusieurs itérations d'hyperparamètres.
- **Ressources GPU limitées** : entraînement sur les GPU **Kaggle**, avec des quotas contraints.

8.2 Manipulation d'un dataset volumineux

Les **80 000 images** de ROCov2 ont soulevé plusieurs défis de volumétrie.

- **Stockage et bande passante** : dépendance à **Hugging Face**, téléchargements non négligeables.
- **Génération des embeddings** : opération longue et coûteuse en mémoire sur les deux modèles.
- **Manque de ressources cloud** : pas de cluster GPU dédié, contraintes mémoire sur le VPS.

8.3 Absence de benchmark de référence en CBIR médical

Aucun **benchmark standardisé** n'existe pour évaluer un système CBIR médical multimodal.

- **Méthodologie ad hoc** : démarche d'évaluation construite à partir des métadonnées.
- **Annotations expertes** : mobilisation de **doctorants en imagerie médicale** pour la vérité terrain.
- **Comparabilité** : difficulté à confronter les résultats à d'autres systèmes.

9 Bilan et retour d'expérience

10 Perspectives

Références bibliographiques

- [1] M. Caron *et al.*, “Emerging Properties in Self-Supervised Vision Transformers,” *ICCV*, 2021.
- [2] S. Zhang *et al.*, “BioMedCLIP : a multimodal biomedical foundation model,” *arXiv :2303.00915*, 2023.
- [3] J. Johnson, M. Douze, H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, 2019.
- [4] FastAPI Documentation. <https://fastapi.tiangolo.com>
- [5] React Documentation. <https://react.dev>

A Guide d'installation et de déploiement

Prérequis

- Python 3.11+, Node.js 20+, Git.
- 4 Go de RAM minimum (index FAISS + modèles chargés en mémoire).
- Clé API Groq pour la synthèse IA.

Installation backend

- [1] Cloner le dépôt : `git clone <url> && cd mediscan-cbir`
- [2] Créer l'environnement virtuel : `python -m venv .venv && source .venv/bin/activate`
- [3] Installer les dépendances : `pip install -r requirements.txt`
- [4] Copier le fichier de configuration : `cp .env.example .env` puis renseigner la clé API Groq et les chemins des index.
- [5] Lancer le serveur : `uvicorn main:app --host 0.0.0.0 --port 8000`

Installation frontend

- [1] Aller dans le répertoire frontend : `cd frontend`
- [2] Installer les dépendances : `npm install`
- [3] Lancer en développement : `npm run dev`
- [4] Ou compiler pour la production : `npm run build`

Déploiement VPS (production)

- Configurer Nginx comme reverse proxy vers `localhost:8000` (backend) et le dossier `dist/` (frontend).
- Créer un service **systemd** pour Gunicorn et l'activer avec `systemctl enable mediscan`.
- Configurer le certificat SSL avec Certbot pour le domaine `mediscan-ai.site`.

B Documentation API REST

Les neuf endpoints de l'API sont exposés sous la base `/api/v1`.

Méthode	Endpoint	Description
POST	/search/visual	Recherche visuelle par image (DINOv2).
POST	/search/semantic	Recherche sémantique par image (BioMedCLIP).
POST	/search/text	Recherche par description textuelle (BioMedCLIP).
POST	/search/multisel	Relance depuis une sélection d'images (centroïde).
GET	/synthesis	Génère la synthèse IA pour un ensemble de résultats.
GET	/export/json	Export des résultats au format JSON.
GET	/export/csv	Export des résultats au format CSV.
GET	/export/pdf	Export des résultats au format PDF.
GET	/health	Vérification de l'état du service.

C Captures d'écran de l'application

D Extraits de code complets

E Résultats de tests détaillés

F Glossaire

CBIR	<i>Content-Based Image Retrieval</i> — recherche d'images par le contenu visuel ou sémantique, sans recours à des métadonnées textuelles manuelles.
Embedding	Représentation vectorielle d'une image ou d'un texte dans un espace de grande dimension, calculée par un réseau de neurones.
FAISS	<i>Facebook AI Similarity Search</i> — bibliothèque de recherche de voisins les plus proches dans des espaces vectoriels de grande dimension.
DINOv2	Modèle de vision auto-supervisé développé par Meta AI, utilisé pour extraire des représentations visuelles de dimension 768.
BioMedCLIP	Modèle multimodal pré-entraîné sur des données biomédicales, alignant images et textes dans un espace vectoriel commun de dimension 512.
Fine-tuning	Adaptation d'un modèle pré-entraîné à un domaine spécifique par un entraînement complémentaire sur des données ciblées.
Top-k	Les k résultats les plus similaires à une requête, retournés par ordre décroissant de similarité.

LLM	<i>Large Language Model</i> — modèle de langage de grande taille, utilisé ici pour générer une synthèse exploratoire à partir des résultats.
Groq	Service d'inférence LLM rapide, utilisé pour la génération de la synthèse assistée.
ROCOv2	<i>Radiology Objects in Context v2</i> — dataset public d'images médicales annotées avec des captions et des concepts UMLS (CUI).
CUI	<i>Concept Unique Identifier</i> — identifiant unique d'un concept dans la nomenclature médicale UMLS.
UMLS	<i>Unified Medical Language System</i> — système de terminologie médicale unifié, référence pour les concepts cliniques.
FastAPI	Framework Python asynchrone pour la création d'API REST, utilisé comme couche applicative backend du système.
React	Bibliothèque JavaScript pour la construction d'interfaces utilisateur, utilisée pour le frontend du système.
Vite	Outil de build et de développement frontend rapide, utilisé pour compiler et servir l'application React.
MongoDB	Base de données orientée documents, utilisée pour l'enrichissement optionnel des métadonnées associées aux résultats de recherche.
Centroïde	Vecteur moyen de plusieurs embeddings, utilisé pour la relance multi-sélection afin de représenter la tendance commune d'un groupe.